

AuditPlatform

Rapport d'analyse — system-design-test2

Généré le 31/03/2026 à 14:55

■ Informations du projet

Nom du projet	system-design-test2
URL du projet	saharchiiha/system-design-test2
Branche analysée	main
Date de l'analyse	31/03/2026 14:22
Modèle LLM	llama-3.3-70b-versatile
Analyse OWASP	Activée

■ Scores

Qualité	52	52/100
Sécurité	20	20/100
Performance	60	60/100

■■ Vulnérabilités (30)

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES
Sévérité : CRITIQUE
Fichier : system-design-primer-master/solutions/object_oriented_design/online_chat/online_chat.py — ligne 1
Correction : Implémenter une authentification sécurisée pour les utilisateurs, avec des mots de passe hashés et des tokens de session sécurisés.
A10 - SSRF (SERVER-SIDE REQUEST FORGERY)

Sévérité : CRITIQUE

Fichier : system-design-primer-master/solutions/system_design/web_crawler/web_crawler_snippets.py — ligne 1

Correction : Valider et sanitizer les URLs pour empêcher les attaques de type SSRF, qui pourraient permettre à un attaquant de contrôler les requêtes HTTP du serveur.

Complexité

Sévérité : HAUTE

Fichier :
system-design-primer-master/solutions/object_oriented_design/call_center/call_center.py — ligne 0

Correction : Réduire la complexité cyclomatique en décomposant les méthodes en sous-méthodes plus simples

Complexité

Sévérité : HAUTE

Fichier : system-design-primer-master/solutions/system_design/mint/mint_mapreduce.py — ligne 0

Correction : Réduire la complexité cyclomatique en décomposant les méthodes en sous-méthodes plus simples

A04 - INSECURE DESIGN

Sévérité : HAUTE

Fichier :
system-design-primer-master/solutions/object_oriented_design/call_center/call_center.py — ligne 1

Correction : Ajouter des mécanismes de sécurité pour les appels, tels que l'authentification et l'autorisation, pour empêcher les accès non autorisés.

A02 - CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : system-design-primer-master/solutions/system_design/mint/mint_snippets.py — ligne 1

Correction : Utiliser des algorithmes de cryptage sécurisés pour protéger les données sensibles, tels que les informations de paiement.

Boucle $O(n^2)$

Sévérité : HAUTE

Fichier :

system-design-primer-master/solutions/object_oriented_design/call_center/call_center.py — ligne 50

Correction : Utiliser une structure de données plus efficace pour stocker les appels en attente, comme une file de priorité, pour réduire la complexité de la recherche d'un employé disponible. Gain estimé : 30% de réduction du temps de traitement

Appel réseau

Sévérité : HAUTE

Fichier :

system-design-primer-master/solutions/system_design/mint/mint_mapreduce.py — ligne 50

Correction : Utiliser un mécanisme de cache pour réduire le nombre d'appels réseau lors de la récupération des données de dépenses. Gain estimé : 40% de réduction du temps de traitement

Appel réseau

Sévérité : HAUTE

Fichier :

system-design-primer-master/solutions/system_design/sales_rank/sales_rank_mapreduce.py — ligne 50

Correction : Utiliser un mécanisme de cache pour réduire le nombre d'appels réseau lors de la récupération des données de ventes. Gain estimé : 40% de réduction du temps de traitement

SOLID-S

Sévérité : HAUTE

Fichier :

system-design-primer-master/solutions/object_oriented_design/call_center/call_center.py — ligne 50

Correction : La classe Employee a plusieurs responsabilités, il faudrait la diviser en plusieurs classes pour respecter le principe de responsabilité unique.

Gestion erreurs

Sévérité : HAUTE

Fichier :

system-design-primer-master/solutions/system_design/mint/mint_mapreduce.py — ligne 30

Correction : Il n'y a pas de gestion des erreurs dans la méthode mapper, il faudrait ajouter des try/except pour gérer les erreurs potentielles.

Duplication

Sévérité : MOYENNE

Fichier :

system-design-primer-master/solutions/object_oriented_design/call_center/call_center.py —
ligne 0

Correction : Supprimer les duplications de code en créant des méthodes communes

Maintenabilité

Sévérité : MOYENNE

Fichier :

system-design-primer-master/solutions/object_oriented_design/hash_table/hash_map.py —
ligne 0

Correction : Améliorer la maintenabilité en utilisant des design patterns et des principes de conception solides

Duplication

Sévérité : MOYENNE

Fichier : system-design-primer-master/solutions/system_design/pastebin/pastebin.py —
ligne 0

Correction : Supprimer les duplications de code en créant des méthodes communes

Maintenabilité

Sévérité : MOYENNE

Fichier : system-design-primer-master/solutions/system_design/sales_rank/sales_rank_map
reduce.py — ligne 0

Correction : Améliorer la maintenabilité en utilisant des design patterns et des principes de conception solides

A05 - SECURITY MISCONFIGURATION

Sévérité : MOYENNE

Fichier : system-design-primer-master/solutions/system_design/pastebin/pastebin.py —
ligne 1

Correction : Configurer correctement les paramètres de sécurité pour les logs et les données stockées, pour éviter les fuites de données.

Mémoire

Sévérité : MOYENNE

Fichier :

system-design-primer-master/solutions/object_oriented_design/call_center/call_center.py — ligne 70

Correction : Utiliser un mécanisme de gestion de la mémoire pour éviter les fuites de mémoire lors de la création et de la suppression d'objets Employé. Gain estimé : 10% de réduction de la consommation de mémoire

Mémoire

Sévérité : MOYENNE

Fichier : system-design-primer-master/solutions/object_oriented_design/deck_of_cards/deck_of_cards.py — ligne 120

Correction : Utiliser un mécanisme de gestion de la mémoire pour éviter les fuites de mémoire lors de la création et de la suppression d'objets Carte. Gain estimé : 10% de réduction de la consommation de mémoire

Mémoire

Sévérité : MOYENNE

Fichier : system-design-primer-master/solutions/system_design/query_cache/query_cache_snippets.py — ligne 100

Correction : Utiliser un mécanisme de gestion de la mémoire pour éviter les fuites de mémoire lors de la création et de la suppression d'objets Node. Gain estimé : 10% de réduction de la consommation de mémoire

Mémoire

Sévérité : MOYENNE

Fichier : system-design-primer-master/solutions/system_design/web_crawler/web_crawler_snippets.py — ligne 100

Correction : Utiliser un mécanisme de gestion de la mémoire pour éviter les fuites de mémoire lors de la création et de la suppression d'objets Page. Gain estimé : 10% de réduction de la consommation de mémoire

Docstring manquant

Sévérité : MOYENNE

Fichier :

system-design-primer-master/solutions/object_oriented_design/call_center/call_center.py — ligne 10

Correction : Ajouter une docstring pour décrire la classe Employee

Nommage ambigu

Sévérité : MOYENNE

Fichier : system-design-primer-master/solutions/system_design/mint/mint_mapreduce.py — ligne 15

Correction : Renommer la variable 'categorizer' en 'categoriseur' pour plus de clarté

SOLID-L

Sévérité : MOYENNE

Fichier : system-design-primer-master/solutions/object_oriented_design/deck_of_cards/deck_of_cards.py — ligne 20

Correction : La classe Card a des méthodes abstraites qui ne sont pas implémentées dans toutes les sous-classes, il faudrait les implémenter pour respecter le principe de substitution de Liskov.

Testabilité

Sévérité : MOYENNE

Fichier : system-design-primer-master/solutions/system_design/web_crawler/web_crawler_mapreduce.py — ligne 10

Correction : Il n'y a pas de tests unitaires pour la classe RemoveDuplicateUrls, il faudrait en ajouter pour améliorer la testabilité.

Lisibilité

Sévérité : FAIBLE

Fichier : system-design-primer-master/solutions/object_oriented_design/deck_of_cards/deck_of_cards.py — ligne 0

Correction : Améliorer la lisibilité en utilisant des noms de variables et de méthodes plus clairs

Lisibilité

Sévérité : FAIBLE

Fichier : system-design-primer-master/solutions/system_design/query_cache/query_cache_snippets.py — ligne 0

Correction : Améliorer la lisibilité en utilisant des noms de variables et de méthodes plus clairs

Boucle O(n)

Sévérité : FAIBLE

Fichier : system-design-primer-master/solutions/object_oriented_design/deck_of_cards/deck_of_cards.py — ligne 100

Correction : Utiliser une structure de données plus efficace pour stocker les cartes, comme un tableau, pour réduire la complexité de la recherche d'une carte spécifique. Gain estimé : 5% de réduction du temps de traitement

Boucle O(n)

Sévérité : FAIBLE

Fichier : system-design-primer-master/solutions/system_design/pastebin/pastebin.py — ligne 30

Correction : Utiliser une structure de données plus efficace pour stocker les URLs, comme un tableau, pour réduire la complexité de la recherche d'une URL spécifique. Gain estimé : 5% de réduction du temps de traitement

Boucle O(n)

Sévérité : FAIBLE

Fichier : system-design-primer-master/solutions/system_design/social_graph/social_graph_snippets.py — ligne 30

Correction : Utiliser une structure de données plus efficace pour stocker les personnes, comme un tableau, pour réduire la complexité de la recherche d'une personne spécifique. Gain estimé : 5% de réduction du temps de traitement

Commentaire absent

Sévérité : FAIBLE

Fichier : system-design-primer-master/solutions/object_oriented_design/deck_of_cards/deck_of_cards.py — ligne 20

Correction : Ajouter un commentaire pour expliquer le but de la classe BlackJackCard

■ Recommandations

1. Utiliser des noms de variables et de méthodes clairs

Utiliser des noms de variables et de méthodes clairs et significatifs pour améliorer la lisibilité du code

2. Décomposer les méthodes en sous-méthodes plus simples

Décomposer les méthodes en sous-méthodes plus simples pour réduire la complexité cyclomatique et améliorer la maintenabilité

3. Supprimer les duplications de code

Supprimer les duplications de code en créant des méthodes communes pour améliorer la maintenabilité et la lisibilité

4. Authentification et autorisation

Implémenter des mécanismes d'authentification et d'autorisation pour les utilisateurs, pour empêcher les accès non autorisés aux ressources sensibles.

5. Cryptage des données

Utiliser des algorithmes de cryptage sécurisés pour protéger les données sensibles, telles que les informations de paiement.

6. Configuration de sécurité

Configurer correctement les paramètres de sécurité pour les logs et les données stockées, pour éviter les fuites de données.

7. Validation des URLs

Valider et sanitiser les URLs pour empêcher les attaques de type SSRF, qui pourraient permettre à un attaquant de contrôler les requêtes HTTP du serveur.

8. Utiliser des structures de données efficaces

Utiliser des structures de données comme des tableaux, des files de priorité, des arbres, etc. pour stocker et récupérer les données de manière efficace.

9. Gérer la mémoire de manière efficace

Utiliser des mécanismes de gestion de la mémoire pour éviter les fuites de mémoire et réduire la consommation de mémoire.

10. Réduire les appels réseau

Utiliser des mécanismes de cache pour réduire le nombre d'appels réseau et améliorer les performances.

11. Améliorer la documentation des classes

Ajouter des docstrings pour décrire les classes et leurs méthodes pour améliorer la compréhension du code

12. Utiliser des noms de variables plus descriptifs

Utiliser des noms de variables plus descriptifs pour améliorer la lisibilité du code

13. Ajouter des commentaires pour expliquer les algorithmes complexes

Ajouter des commentaires pour expliquer les algorithmes complexes pour améliorer la compréhension du code

14. Refactorisation de la classe Employee

Il faudrait diviser la classe Employee en plusieurs classes pour respecter le principe de responsabilité unique. Par exemple, une classe pour les opérateurs, une classe pour les superviseurs, etc.

15. Implémentation des méthodes abstraites

Il faudrait implémenter les méthodes abstraites de la classe Card dans toutes les sous-classes pour

respecter le principe de substitution de Liskov.

16. Gestion des erreurs

Il faudrait ajouter des try/except dans les méthodes pour gérer les erreurs potentielles et améliorer la robustesse du code.

17. Ajout de tests unitaires

Il faudrait ajouter des tests unitaires pour les classes et les méthodes pour améliorer la testabilité et la qualité du code.